

ecperf : Performance data gathering ...

... with (ec)perf and result processing using SQLite

Sami Saarinen

sami.saarinen@iki.fi

HPC-consulting Ltd, Espoo, Finland



Overview

- Talking points
- perf and ecp perf
- Workflows
- Example cases
- DrHook-outputs
- ecp erf-stat outputs
- ecp erf-digest outputs
- Working with SQLite database
- Supplementary slides

Talking points

- Performance profiling using the `perf`-tool
- Unlike with the `DrHook` there is no need to add explicit (profiling) function calls to the code
- *Sampling*: program execution gets interrupted at regular intervals to collect performance information from the requested software and hardware counters
- The `perf`-tool allows very flexible access to calling tree (call-graph) information -- with some caveat in the performance
- Allows to discover performance beyond the executable (e.g. IFS) boundaries, into the library routines (MPI, OpenMP, mathlibs, MKL, ...)

Talking points ... cont'd

- Since performance data is complex and its volume is huge, consider the perf-approach “expert-only” : certainly too expensive to enable it for all the jobs (or even all the tasks)
- Some new tools will be shown how to place performance information into the SQLite database for repeated queries, deeper analysis and time-series plots
- Please note that DrHook explicit calls still allow more accurate results than perf-tool, but is
 - limited to functions that are being explicitly made to be monitored by IFS,
 - only cheap to obtain information about wall-clock times (per thread),
 - and rather expensive to produce calling trees
- It is interesting to compare performance between DrHook and (ec)perf runs

A quick intro to the perf-tool

- Implements a family of subcommands, of which the **highlighted** ones are of a great interest now
 - Unfortunately some of the subcommands require root-privileges

```
perf help
```

```
usage: perf [--version] [--help] [OPTIONS] COMMAND [ARGS]
```

The most commonly used perf commands are:

annotate	Read perf.data (created by perf record) and display annotated code
archive	Create archive with object files with build-ids found in perf.data file
bench	General framework for benchmark suites
buildid-cache	Manage build-id cache.
buildid-list	List the buildids in a perf.data file
c2c	Shared Data C2C/HITM Analyzer.
config	Get and set variables in a configuration file.
data	Data file related processing
diff	Read perf.data files and display the differential profile
evlist	List the event names in a perf.data file
ftrace	simple wrapper for kernel's ftrace functionality
inject	Filter to augment the events stream with additional information
kallsyms	Searches running kernel for symbols
kmem	Tool to trace/measure kernel memory properties
kvm	Tool to trace/measure kvm guest os
list	List all symbolic event types
lock	Analyze lock events
mem	Profile memory accesses
record	Run a command and record its profile into perf.data
report	Read perf.data (created by perf record) and display the profile
sched	Tool to trace/measure scheduler properties (latencies)
script	Read perf.data (created by perf record) and display trace output
stat	Run a command and gather performance counter statistics
test	Runs sanity tests.
timechart	Tool to visualize total system behavior during a workload
top	System profiling tool.
version	display the version of perf binary
probe	Define new dynamic tracepoints
trace	strace inspired tool

See 'perf help COMMAND' for more information on a specific command.

What is the `ecperf` ?

- At present it is a `perf`-tool executable built from Linux kernel sources, currently the version 5.14
- Can (at least) be run on Linux kernels 4.18 or later
- Brings hugely improved performance in function name resolution
 - Speed ups from hours to minutes or even to tens of seconds
 - Reason: use of `addr2line`-utility underneath is optimized
- Makes a sampling based performance profiling of large multi-threaded applications feasible
- Each MPI-task (with its threads) can be profiled independently
- Threads may not even (all) belong to the OpenMP-threads:
some MPI-libs implement extra threads as progress engines (e.g. OpenMPI)

Overcoming issues with the default perf

- The standard perf (prior to circa version 5.11) can be very slow to resolve large amounts of function symbols from program counter information
- Obtaining realistic call-graph statistics can take many orders of magnitude longer to resolve than running the actual application itself
- However, it is possible to obtain the **source code** of a newer (kernel) version from

<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git>

- .. and build a subset of it from the scratch
- For example, the following commands build the version 5.14 and perf gets renamed to ecperf

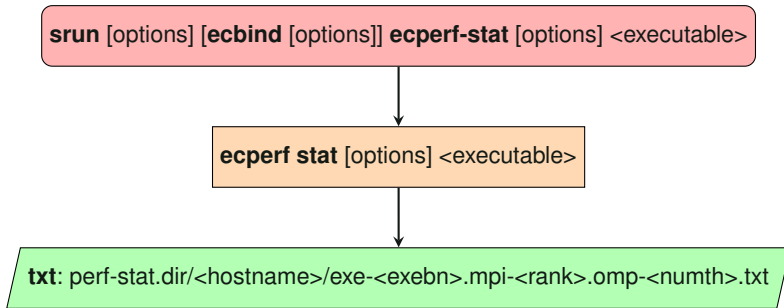
```
% tar xf linux-5.14.tar.gz
% cd linux-5.14/tools/perf
% make JOBS=$(nproc --all) # A parallel make ok, but not invoked via -j
% ldd perf
% cp perf $HOME/.local/bin/ecperf
```

Workflows

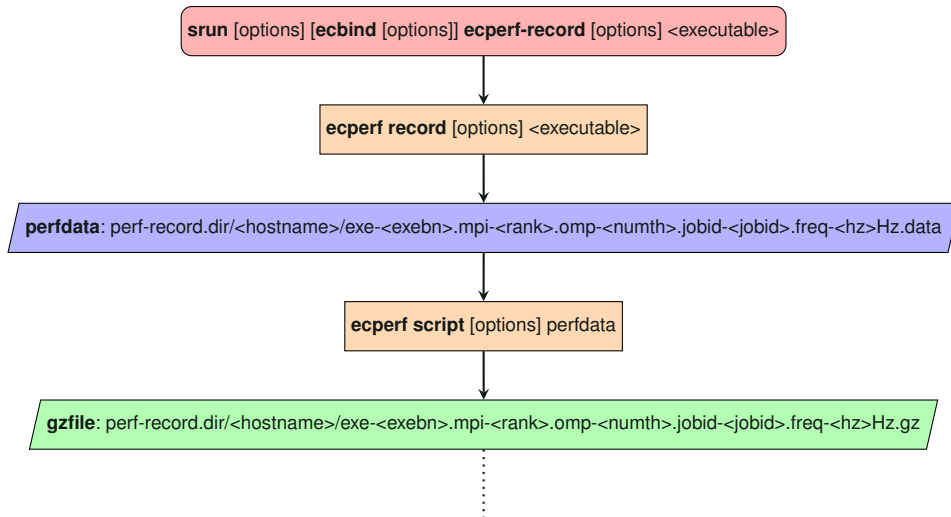
Tools developed to control `ecperf` workflows

- `ecperf-stat`
 - Collects by default performance counter information from all MPI-ranks
- `ecperf-record`
 - Records the requested H/W or S/W counter events by sampling at regular intervals
 - Supports selection of performance groups, events and subsets per MPI-task basis
 - Recording can be started after a delay and **perfddata** output size can be capped
 - The **perfddata** will be converted into **gzfile** using custom **ecperf script** command
- `ecperf-digest`
 - Reads **gzfile** and creates **dbfile** (per MPI-rank) which is **sqlite3**-compatible database
 - Creates performance profiles and annotated profiles (with embedded source code)
 - Can read previously created **dbfile** directly
 - As a result performance/annotated profiles are superfast to **re-create** & explore
 - **dbfile** can also be manipulated directly with **sqlite3** interfaces (CLI, Perl, Python, ...)
 - Predefined SQL-views (**CREATE VIEW**) simplify making queries, f.ex. ...
 - **SELECT * FROM callproflines WHERE event_name = 'cycles' LIMIT 10;**
 - ... produces top-10 calling trees from sampling events called 'cycles'

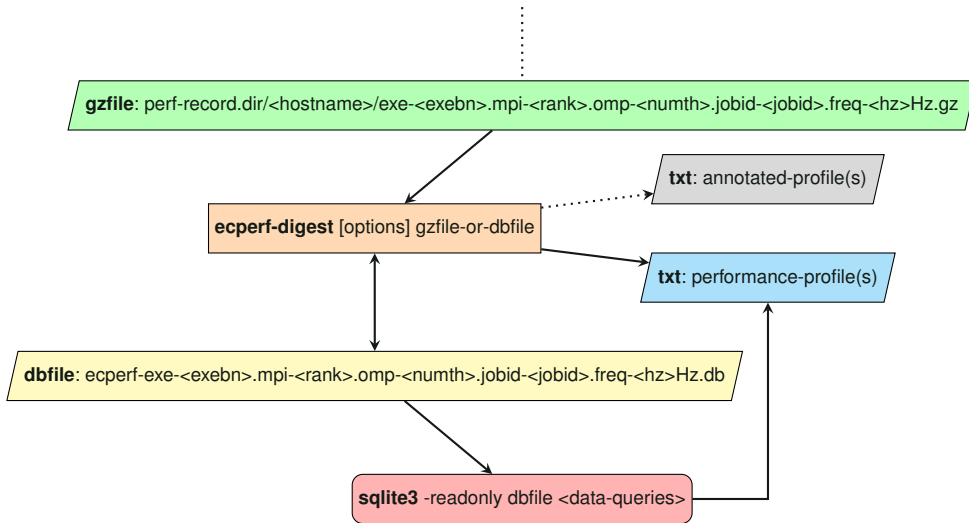
ecperf-stat workflow



ecperf-record workflow



ecperf-digest workflow



Example cases

- IFS RAPS20 development version (branch `naih_CY47R3_RAPS20`):
CY47R3+
- Ran TCo399L137 HRES 10-day forecast (operational output)
- 9 compute nodes with 8-way OpenM and one I/O-server node (all 16 MPI-tasks/node)
- Used Intel compiler 2021.2.0 with MKL 19.5 over HPCX 2.9.0 OpenMPI libraries
- Several, separate runs to get output from
 - DrHook
 - `ecperf-stat`
 - `ecperf-record` – 4 runs with different sets of performance event counters
- Checking the first compute MPI-rank (#0),
and the first I/O-server rank (#128) (here: DrHook only)

DrHook-output summaries

■ Across all the compute ranks:

```
The name of the executable : /lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP
Number of MPI-tasks       : 128
Number of OpenMP-threads  : 8
export THRESHOLD_PERCENT  : 0.1%
Wall-times over 128 MPI-tasks (secs) : Min=1714.450, Max=1720.460, Avg=1716.508, StDev=1.157
Routines whose average time > 0.10% (1.717 secs) of the total average will be included in the listing
Avg-% Avg.secs Min.secs Max.secs St.dev Imbal-% # of calls Avg.msec/call Tasks OpenMP : Name-of-the-routine
7.64% 131.164 35.784 154.582 36.063 76.85% 187008 0.701 128 1 : >MPL-TRLTOM_COMMS(806)
4.88% 83.737 23.594 118.218 20.005 80.04% 128000 0.654 128 1 : >BAR-BARRIERINWRMLFP(792)
4.04% 69.313 59.967 90.197 5.049 33.52% 77952 0.889 128 1 : >BAR-BARRIERINPOSDDH(799)
2.83% 48.564 45.094 52.047 1.342 13.36% 39362560 0.001 128 8 : CLOUDSC
2.71% 46.467 45.945 47.367 0.317 3.00% 30848 1.506 128 1 : UPDECAEC
2.37% 40.661 18.548 137.710 34.898 86.53% 374016 0.109 128 1 : >MPL-TRGTOL_COMMS(803)
1.70% 29.231 29.209 29.241 0.007 0.11% 92160 0.317 128 1 : sol_pcg
1.69% 29.082 28.692 29.364 0.125 2.29% 92160 0.316 128 1 : dia_vri
1.41% 24.179 23.308 24.948 0.338 6.57% 78725120 0.000 128 8 : LAITRI_WENO
1.37% 23.436 7.845 50.575 9.197 84.49% 5120000 0.005 128 8 : RTTOV_INTEGRATE
1.33% 22.888 21.332 23.951 0.675 10.93% 196812800 0.000 128 8 : LARCHE
1.33% 22.879 22.043 23.812 0.351 7.43% 92160 0.248 128 1 : tra_adv_tvd
1.29% 22.091 19.177 25.391 1.165 24.47% 3269888 0.007 128 8 : radiation_mcica_sw:solver_mcica_sw
1.24% 21.348 21.145 21.539 0.092 1.83% 92160 0.232 128 1 : tra_ldf_iso
1.16% 19.975 19.665 20.257 0.102 2.92% 92160 0.217 128 1 : ldf_slp
```

■ Across all the I/O ranks:

```
The name of the executable : /lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP
Number of MPI-tasks       : 16
Number of OpenMP-threads  : 1
export THRESHOLD_PERCENT  : 0.1%
Wall-times over 16 MPI-tasks (secs) : Min=1707.050, Max=1707.070, Avg=1707.062, StDev=0.005
Routines whose average time > 0.10% (1.707 secs) of the total average will be included in the listing
Avg-% Avg.secs Min.secs Max.secs St.dev Imbal-% # of calls Avg.msec/call Tasks OpenMP : Name-of-the-routine
81.36% 1388.829 1366.288 1409.262 12.886 3.05% 2456080 0.565 16 1 : IO_SERV_RECV_REQ:PROBEX
4.86% 82.964 58.150 107.264 14.658 45.79% 511583 0.162 16 1 : GRIB_API:IGRIB_SET_REAL4_ARRAY
3.26% 55.636 53.813 57.225 1.013 5.96% 2196000 0.025 16 1 : IO_SERV_RECV_SORT
3.19% 54.446 48.017 59.714 4.434 19.59% 401333 0.136 16 1 : IWRITEFDBSUBS
1.49% 25.457 23.148 28.935 1.706 20.00% 2032 12.528 16 1 : IFLUSHFDBSUBS
1.14% 19.411 19.408 19.415 0.003 0.04% 16 1213.207 16 1 : IO_SERV_RECV_REQ:PROBE1
1.11% 18.916 18.393 19.429 0.286 5.33% 2456096 0.008 16 1 : IO_SERV_GET_REQ
0.78% 13.390 13.292 13.477 0.065 1.37% 110250 0.121 16 1 : WGRIBENCODE
0.59% 10.061 6.857 12.234 1.584 43.95% 2016 4.991 16 1 : IO_SERV_CLOSE_EC
```

DrHook-output from the first compute rank

Profiling information for program='/lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP', proc#1:

```
No. of instrumented routines called : 1586
Instrumentation started : 20211122 111055
Instrumentation ended : 20211122 113922
Instrumentation overhead: 0.36%
Memory usage : 13983 MB (heap), 7721 MB (rss), 169 MB (stack), 47170 MB (vmpeak), 0 (paging)
Wall-time is 1718.94 sec on proc#1 (144 procs, 8 threads)
Thread#1: 1701.27 sec (98.97%) -- Cycles: 3841596483524 (3841881508328)
Thread#2: 770.06 sec (44.80%) -- Cycles: 1744440619686 (3841881375713)
Thread#3: 769.86 sec (44.79%) -- Cycles: 1744444299011 (3841881436530)
Thread#4: 770.00 sec (44.79%) -- Cycles: 1744431166157 (3841881423683)
Thread#5: 770.01 sec (44.80%) -- Cycles: 1744429329160 (3841881410070)
Thread#6: 769.99 sec (44.79%) -- Cycles: 1744433694743 (3841881320745)
Thread#7: 769.76 sec (44.78%) -- Cycles: 1744430748037 (3841881265665)
Thread#8: 770.13 sec (44.80%) -- Cycles: 1744425863335 (3841881521130)
```

#	% Time	Cumul	Self	Total	# of calls	Self	Total	Routine@<thread-id>
	(self)	(sec)	(sec)	(sec)		(Size; ms/call)	(Size; ms/call)	(Size/sec; Size/call; MinSize; MaxSize)
1	5.70	98.039	98.039	98.039	824	118.98	118.98	GRIB_API:IGRIB_SET_REAL4_ARRAY@1
2	4.99	183.885	85.846	85.847	609	140.96	140.96	>BAR-BARRIER IN POSDDH (799)@1
3	4.62	263.334	79.449	79.449	1000	79.45	79.45	>BAR-BARRIER IN WRMLFP (792)@1
4	2.78	311.106	47.772	54.274	38220	1.25	1.42	*CLOUDSC@8
5	2.75	311.106	47.298	53.919	38974	1.21	1.38	CLOUDSC@3
6	2.75	311.106	47.292	53.811	38988	1.21	1.38	CLOUDSC@2
7	2.74	311.106	47.155	53.665	39284	1.20	1.37	CLOUDSC@1
8	2.71	311.106	46.598	53.003	37961	1.23	1.40	CLOUDSC@4
9	2.70	311.106	46.375	52.931	37961	1.22	1.39	CLOUDSC@7
10	2.70	357.446	46.340	46.347	241	192.28	192.31	UPDECAEC@1
11	2.70	357.446	46.329	52.794	38059	1.22	1.39	CLOUDSC@5
12	2.69	357.446	46.305	52.737	38073	1.22	1.39	CLOUDSC@6
13	2.19	395.064	37.619	37.620	1461	25.75	25.75	>MPL-TRLTOM_COMMS (806)@1
14	1.70	424.284	29.219	29.221	720	40.58	40.58	sol_pcg@1
15	1.70	453.483	29.199	29.201	720	40.55	40.56	dia_wri@1
16	1.65	481.887	28.404	28.405	2922	9.72	9.72	>MPL-TRGTOL_COMMS (803)@1
17	1.65	510.235	28.349	28.362	4872	5.82	5.82	*RTTOV_INTEGRATE@8
18	1.64	510.235	28.201	28.221	5011	5.63	5.63	RTTOV_INTEGRATE@2
19	1.64	510.235	28.178	28.199	4978	5.66	5.66	RTTOV_INTEGRATE@7
20	1.64	510.235	28.145	28.171	5010	5.62	5.62	RTTOV_INTEGRATE@3
21	1.64	510.235	28.131	28.149	5016	5.61	5.61	RTTOV_INTEGRATE@6
22	1.63	510.235	28.087	28.114	5065	5.55	5.55	RTTOV_INTEGRATE@1
23	1.63	510.235	27.983	28.003	5027	5.57	5.57	RTTOV_INTEGRATE@5
24	1.63	510.235	27.964	27.984	5021	5.57	5.57	RTTOV_INTEGRATE@4

DrHook-output from the first I/O-server rank

Profiling information for program='/lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP', proc#129:

No. of instrumented routines called : 66
 Instrumentation started : 20211122 111055
 Instrumentation ended : 20211122 113922
 Instrumentation overhead: 0.01%
 Memory usage : 1019 MB (heap), 845 MB (rss), 2 MB (stack), 34205 MB (vmpeak), 0 (paging)
 Wall-time is 1707.05 sec on proc#129 (144 procs, 1 threads)
 Thread#1: 1707.05 sec (100.00%) -- Cycles: 3841252099808 (3841369433932)

#	% Time	Cumul	Self	Total	# of calls	Self	Total	Routine@<thread-id>
	(self)	(sec)	(sec)	(sec)		ms/call	ms/call	(Size; Size/sec; Size/call; MinSize; MaxSize)
1	82.25	1403.965	1403.965	1404.020	168255	8.34	8.34	IO_SERV_RECV_REQ:PROBEX01
2	3.56	1464.717	60.752	60.760	32750	1.86	1.86	GRIB_API:IGRIB_SET_REAL4_ARRAY01
3	3.50	1524.431	59.714	59.727	25750	2.32	2.32	IWRITEFDBSUBS01
4	3.35	1581.656	57.225	77.126	152000	0.38	0.51	IO_SERV_RECV_SORT01
5	1.57	1608.530	26.874	26.875	127	211.61	211.61	IFLUSHFDBSUBS01
6	1.14	1627.959	19.429	19.907	168256	0.12	0.12	IO_SERV_GET_REQ01
7	1.14	1647.373	19.414	19.414	1	19413.75	19414.28	IO_SERV_RECV_REQ:PROBE101
8	0.79	1660.779	13.406	31.207	7000	1.92	4.46	WGRIBENC001
9	0.53	1669.758	8.979	35.853	126	71.26	284.55	IO_SERV_CLOSE_EC01
10	0.52	1678.699	8.941	8.945	25750	0.35	0.35	GRIB_API:IGRIB_CLONE01
11	0.39	1685.360	6.661	6.674	168255	0.04	0.04	IO_SERV_RECV_REQ:RECVX01
12	0.28	1690.184	4.824	5.033	1	4824.49	5032.86	IO_SERV_SUIOSCTMPL01
13	0.27	1694.859	4.674	4.688	25750	0.18	0.18	GRIB_API:IGRIB_SET_MESSAGE01
14	0.20	1698.266	3.407	3.415	116944	0.03	0.03	GRIB_API:IGRIB_SET_CHAR01
15	0.17	1701.188	2.922	3.647	18750	0.16	0.19	GRIB_UTILS_MOD:GRIB_SET_PACKING01
16	0.09	1702.780	1.593	1.604	191400	0.01	0.01	GRIB_API:IGRIB_SET_INT01
17	0.07	1703.981	1.200	1.203	25750	0.05	0.05	GRIB_API:IGRIB_RELEASE01
18	0.04	1704.617	0.636	0.637	7000	0.09	0.09	FITSPECTRUM01
19	0.03	1705.076	0.459	0.469	168256	0.00	0.00	IO_SERV_GET_REQID01
20	0.02	1705.390	0.314	1507.690	1752	0.18	860.55	IO_SERV_RECV_REQ01
21	0.01	1705.627	0.237	106.817	18750	0.01	5.70	IO_SERV_WRITEFLD_EC01
22	0.01	1705.826	0.199	31.406	7000	0.03	4.49	IO_SERV_WV_WGRIBENC001
23	0.01	1706.010	0.184	0.184	7	26.34	26.34	GRIB_API:IGRIB_NEW_FROM_MESSAGE01
24	0.01	1706.178	0.167	41.379	7000	0.02	5.91	IO_SERV_WV_REC01
25	0.01	1706.334	0.157	158.505	1625	0.10	97.54	IO_SERV_WRITE_EC01
26	0.01	1706.448	0.114	0.115	14000	0.01	0.01	GRIB_API:IGRIB_GET_INT01

ecperf-stat outputs

- Output were obtained on every MPI-task by applying ecperf-stat script to the IFS executable through srun as follows:

```
srun -l -v -K1 --cpus-per-task=8 --ntasks-per-node=16 --threads-per-core=1 \  
--hint=nomultithread --cpu-bind=none -N9 -n 144 \  
ecbind -V0 --ppn 16 -h 1 --depth 8 --bind=2.8 --places --cps 64 \  
--tool ecperf-stat /lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP
```

- The actual (lengthy) argument list (inside the ecperf-stat-script) was as follows (output is for the very first MPI-rank#0):

```
ecperf stat -e task-clock,context-switches,cpu-migrations,page-faults,cycles,\  
stalled-cycles-frontend,stalled-cycles-backend,instructions,\  
branches,branch-misses,\  
L1-dcache-loads,L1-dcache-load-misses,\  
cache-references,\  
LLC-loads,LLC-load-misses,\  
L1-icache-loads,L1-icache-load-misses,\  
dTLB-loads,dTLB-load-misses,iTLB-loads,iTLB-load-misses,\  
L1-dcache-prefetches,L1-dcache-prefetch-misses,\  
fp_ret_sse_avx_ops.all --delay 0 \  
-o /the/rundir/perf-stat.dir/aa1-3048/exe-ifsMASTER.SP.mpi-0.omp-8.txt -- \  
/lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP
```

ecperf-stat outputs : the first compute rank

started on Mon Nov 22 11:10:43 2021

Performance counter stats for '/lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP':

13,732,562.70 msec	task-clock	#	7.914 CPUs utilized	
421,038	context-switches	#	30.660 /sec	
19	cpu-migrations	#	0.001 /sec	
21,515,599	page-faults	#	1.567 K/sec	
42,652,500,129,523	cycles	#	3.106 GHz	(29.41%)
3,999,346,342,672	stalled-cycles-frontend	#	9.38% frontend cycles idle	(29.41%)
27,389,074,914,075	stalled-cycles-backend	#	64.21% backend cycles idle	(29.41%)
43,884,709,819,067	instructions	#	1.03 insn per cycle	
		#	0.62 stalled cycles per insn	(29.41%)
7,057,302,144,274	branches	#	513.910 M/sec	(29.41%)
27,740,893,528	branch-misses	#	0.39% of all branches	(35.30%)
21,046,637,191,342	L1-dcache-loads	#	1.533 G/sec	(35.29%)
579,670,622,237	L1-dcache-load-misses	#	2.75% of all L1-dcache accesses	(35.29%)
989,714,421,656	cache-references	#	72.071 M/sec	(35.29%)
<not supported>	LLC-loads			
<not supported>	LLC-load-misses			
1,086,089,377,605	L1-icache-loads	#	79.089 M/sec	(35.29%)
8,072,587,915	L1-icache-load-misses	#	0.74% of all L1-icache accesses	(35.29%)
51,131,962,000	dTLB-loads	#	3.723 M/sec	(35.29%)
8,859,817,623	dTLB-load-misses	#	17.33% of all dTLB cache accesses	(35.29%)
1,478,905,467	iTLB-loads	#	107.693 K/sec	(35.29%)
300,635,203	iTLB-load-misses	#	20.33% of all iTLB cache accesses	(35.29%)
354,603,691,539	L1-dcache-prefetches	#	25.822 M/sec	(35.29%)
<not supported>	L1-dcache-prefetch-misses			
12,795,445,197,272	fp_ret_sse_avx_ops.all	#	931.759 M/sec	(29.41%)

1735.255985853 seconds time elapsed

12129.541141000 seconds user

1407.049059000 seconds sys

ecperf-stat outputs : the first I/O-server rank

started on Mon Nov 22 11:10:43 2021

Performance counter stats for '/lus/h2resw01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP':

1,706,114.09 msec	task-clock	#	0.983 CPUs utilized	
43,117	context-switches	#	25.272 /sec	
16	cpu-migrations	#	0.009 /sec	
537,796	page-faults	#	315.217 /sec	
5,764,289,999,159	cycles	#	3.379 GHz	(29.41%)
47,081,239,142	stalled-cycles-frontend	#	0.82% frontend cycles idle	(29.41%)
440,937,745,965	stalled-cycles-backend	#	7.65% backend cycles idle	(29.41%)
14,632,676,278,209	instructions	#	2.54 insn per cycle	
		#	0.03 stalled cycles per insn	(29.41%)
3,141,677,502,815	branches	#	1.841 G/sec	(29.41%)
1,998,131,649	branch-misses	#	0.06% of all branches	(35.30%)
7,195,980,104,909	L1-dcache-loads	#	4.218 G/sec	(35.30%)
22,142,274,906	L1-dcache-load-misses	#	0.31% of all L1-dcache accesses	(35.29%)
46,504,712,397	cache-references	#	27.258 M/sec	(35.29%)
<not supported>	LLC-loads			
<not supported>	LLC-load-misses			
97,980,217,200	L1-icache-loads	#	57.429 M/sec	(35.30%)
834,709,155	L1-icache-load-misses	#	0.85% of all L1-icache accesses	(35.30%)
1,414,287,259	dTLB-loads	#	828.952 K/sec	(35.29%)
313,481,626	dTLB-load-misses	#	22.17% of all dTLB cache accesses	(35.29%)
184,412,723	iTLB-loads	#	108.089 K/sec	(35.29%)
11,019,096	iTLB-load-misses	#	5.98% of all iTLB cache accesses	(35.29%)
13,791,272,959	L1-dcache-prefetches	#	8.083 M/sec	(35.29%)
<not supported>	L1-dcache-prefetch-misses			
80,740,409,697	fp_ret_sse_avx_ops.all	#	47.324 M/sec	(29.41%)

1734.811772829 seconds time elapsed

1667.613667000 seconds user

26.306753000 seconds sys

ecperf-digest : compute rank#0 databases

- We have databases for the 1st rank available from 4 separate runs
- Metadata in the TABLE **mpi** (and **event**) can show useful information
- The following ecperf-digest-command proves to be very useful:

```
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-*.freq-17Hz.db --mode line \  
--select 'database_filename,runtime,num_event_ids,(select group_concat(name) from event) as event_names from mpi'  
SELECT database_filename,runtime,num_event_ids,(select group_concat(name) from event) as event_names from mpi;  
database_filename = /zfs1/mps/BLQ/perffi/22nov2021/hres_tco399/intel.blq.hpcx/ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364585.freq-17Hz.db  
runtime = 2287.57356999401  
num_event_ids = 9  
event_names = cycles,instructions,L1-dcache-loads,L1-dcache-load-misses,cache-references,dTLB-load-misses,branch-misses,branch-instructions,task-clock  
database_filename = /zfs1/mps/BLQ/perffi/22nov2021/hres_tco399/intel.blq.hpcx/ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db  
runtime = 1769.42398758396  
num_event_ids = 3  
event_names = cycles,instructions,task-clock  
database_filename = /zfs1/mps/BLQ/perffi/22nov2021/hres_tco399/intel.blq.hpcx/ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364628.freq-17Hz.db  
runtime = 1826.51682924101  
num_event_ids = 2  
event_names = L1-dcache-loads,L1-dcache-load-misses  
database_filename = /zfs1/mps/BLQ/perffi/22nov2021/hres_tco399/intel.blq.hpcx/ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364684.freq-17Hz.db  
runtime = 1850.742319001  
num_event_ids = 3  
event_names = cache-references,dTLB-load-misses,faults
```

- By asking too many events (like 9 in one of the runs) can increase the runtime (ecperf overhead) considerably
- The next couple of slides some of these databases are examined
btw: these were moved to the local Linux desktop first!

ecperf-digest : compute rank#0, basic event group: cycles, line numbers (with source filenames)

```
# h240.N8+1.T128xt8xh1+ioT16xt8.iomode.od.i16r0w24.0RCA025.Z75.htco399-1364623.od-rec-basic-17hz/perf-record.dir/aa2-4036/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db --linenos --nocalltree --top 5 -e cycles --stdio --verbose
# [...] : Time = 1769s (1769s): Displaying top-5 (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs

# [...] : Time = 1769s (1769s): Total number of #Events:cycles = 43,160,625,032,310 (2711M #Ev/sec/thread) in 237619 samples over 9 threads at 14 Hz (nominal 17 Hz)
# (30x10) : SELECT top,prc,mevs,time,nevents,nsamples,tid,cpulist,scope,code_location FROM proflines WHERE event_name = 'cycles' AND top <= 5 ORDER BY top,nevents DESC,time DESC
#Top  Event-% [Time] #Events:cycles (#Ev/sec) | Samples TID CPU-list Scope Code location
#----
1 38.01% 640.6s 2,054,544,753,572 ( 3207M ) | 10945 2495675 6 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
| 37.66% 634.2s 2,033,628,937,136 ( 3206M ) | 10841 2495695 7
| 37.41% 630.7s 2,020,382,975,524 ( 3204M ) | 10778 2495626 3
| 37.24% 627.1s 2,012,651,181,964 ( 3209M ) | 10724 2495641 4
| 37.11% 625.3s 2,003,679,215,320 ( 3205M ) | 10686 2495611 2
| 36.99% 622.8s 1,999,053,393,692 ( 3210M ) | 10642 2495657 5
| 32.02% 539.2s 1,729,511,402,535 ( 3208M ) | 9214 2495703 0
2 9.94% 175.3s 531,295,080,931 ( 3031M ) | 2823 2495483 0-2,4 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0)
| 9.02% 153.1s 487,358,673,238 ( 3184M ) | 2586 2495675 6
| 9.01% 152.8s 486,742,614,902 ( 3186M ) | 2591 2495641 4
| 8.91% 151.3s 481,158,598,445 ( 3180M ) | 2562 2495703 0
| 8.89% 150.8s 480,227,052,354 ( 3185M ) | 2554 2495695 7
| 8.86% 150.3s 478,981,385,948 ( 3186M ) | 2546 2495657 5
| 8.86% 150.3s 478,587,711,329 ( 3185M ) | 2543 2495626 3
| 8.80% 149.1s 475,185,068,882 ( 3187M ) | 2524 2495611 2
3 5.51% 93.05s 297,500,785,704 ( 3197M ) | 1587 2495703 0 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
| 0.03% 466ms 1,440,039,420 ( 3092M ) | 8 2495695 7
| 0.02% 297ms 969,808,311 ( 3270M ) | 5 2495641 4
| 0.01% 165ms 505,720,507 ( 3056M ) | 3 2495657 5
| 0.01% 126ms 394,342,130 ( 3131M ) | 2 2495626 3
| 0.01% 117ms 386,980,727 ( 3299M ) | 2 2495611 2
| 0.01% 110ms 317,759,645 ( 2881M ) | 2 2495675 6
4 3.18% 53.50s 171,887,036,375 ( 3213M ) | 916 2495611 2 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
| 3.16% 53.14s 170,561,129,984 ( 3210M ) | 907 2495626 3
| 3.14% 52.89s 169,574,968,878 ( 3206M ) | 902 2495695 7
| 3.13% 52.65s 169,236,594,926 ( 3215M ) | 902 2495641 4
| 3.12% 52.32s 168,393,750,208 ( 3219M ) | 896 2495657 5
| 3.10% 52.28s 167,322,043,277 ( 3201M ) | 893 2495703 0
| 2.99% 50.30s 161,605,785,643 ( 3213M ) | 860 2495675 6
5 2.95% 49.93s 157,675,174,487 ( 3158M ) | 846 2495483 1 PTHREAD pthread_spin_lock @ libpthread-2.28.so (???):0)
```

ecperf-digest : compute rank#0, basic event group: cycles, linenos, code location replication

```
# h240.N8+1.T128xt8xhi+ioT16xt8.iomode.od.i16r0w24.URCA025.Z75.htco399-1364623.od-rec-basic-17hz/perf-record.dir/aa2-4036/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db --linenos --nocalltree --top 5 -e cycles --replicate --less -v
# [...] : Time = 1769s (1769s): Displaying top=5 (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs

# [...] : Time = 1769s (1769s): Total number of #Events:cycles = 43,160,625,032,310 (2711M #Ev/sec/thread) in 237619 samples over 9 threads at 14 Hz (nominal 17 Hz)
# (30x10) : SELECT top,prc,mevs,time,nevents,nsamples,tid,cpulist,scope,code_location FROM proflines WHERE event_name = 'cycles' AND top <= 5 ORDER BY top,nevents DESC,time DESC
#Top  Event-% [Time] #Events:cycles (#Ev/sec) | Samples TID CPU-list Scope Code location
#----
1 38.01% 640.6s 2,054,544,753,572 (3207M) | 10945 2495675 6 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
1 37.66% 634.2s 2,033,628,937,136 (3206M) | 10841 2495695 7 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
1 37.41% 630.7s 2,020,382,975,524 (3204M) | 10778 2495626 3 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
1 37.24% 627.1s 2,012,651,181,964 (3209M) | 10724 2495641 4 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
1 37.11% 625.3s 2,003,679,215,320 (3205M) | 10686 2495611 2 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
1 36.99% 622.8s 1,999,053,393,692 (3210M) | 10642 2495657 5 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
1 32.02% 539.2s 1,729,511,402,535 (3208M) | 9214 2495703 0 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:596)
2 9.94% 175.3s 531,295,080,931 (3031M) | 2823 2495483 0-2,4 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 9.02% 153.1s 487,358,673,238 (3184M) | 2586 2495675 6 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 9.01% 152.8s 486,742,614,902 (3186M) | 2591 2495641 4 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 8.91% 151.3s 481,158,598,445 (3180M) | 2562 2495703 0 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 8.89% 150.8s 480,227,052,354 (3185M) | 2554 2495695 7 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 8.86% 150.3s 478,981,385,948 (3186M) | 2546 2495657 5 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 8.86% 150.3s 478,587,711,329 (3185M) | 2543 2495626 3 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
2 8.80% 149.1s 475,185,068,882 (3187M) | 2524 2495611 2 KERNEL native_queued_spin_lock_slowpath @ ??? (???):0
3 5.51% 93.05s 297,500,785,704 (3197M) | 1587 2495703 0 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
3 0.03% 466ms 1,440,039,420 (3092M) | 8 2495695 7 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
3 0.02% 297ms 969,808,311 (3270M) | 5 2495641 4 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
3 0.01% 165ms 505,720,507 (3056M) | 3 2495657 5 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
3 0.01% 129ms 394,342,130 (3131M) | 2 2495626 3 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
3 0.01% 117ms 386,980,727 (3209M) | 2 2495611 2 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
3 0.01% 110ms 317,759,645 (2881M) | 2 2495675 6 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:587)
4 3.18% 53.50s 171,887,036,375 (3213M) | 916 2495611 2 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
4 3.16% 53.14s 170,561,129,984 (3210M) | 907 2495626 3 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
4 3.14% 52.89s 169,574,968,878 (3206M) | 902 2495695 7 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
4 3.13% 52.65s 169,236,594,926 (3215M) | 902 2495641 4 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
4 3.12% 52.32s 168,393,750,208 (3219M) | 896 2495657 5 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
4 3.10% 52.28s 167,322,043,277 (3201M) | 893 2495703 0 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
4 2.99% 50.30s 161,605,785,643 (3213M) | 860 2495675 6 OMP _INTERNALc1e8d79f::__kmp_wait_template @ libiomp5.so (kmp_wait_release.h:553)
5 2.95% 49.93s 157,675,174,487 (3158M) | 846 2495483 1 PTHREAD pthread_spin_lock @ libpthread-2.28.so (???):0
```

ecperf-digest : compute rank#0, basic event group: DrHook-like task-clock

```
# h240.N8+1.T128xt8sh1+ioT16xt8.iomode.od.i16r0w24.DRCAO25.Z75.htco399-1364623.od-rec-basic-17hz/perf-record.dir/aa2-4036/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db -e task-clock --where 'scope like "%user%"' --limit 30 --replicate -v
# [...] : Time = 1769s (1769s): Displaying (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs up to 30 rows/output

# [...] : Time = 1769s (1769s): Total number of #Events:task-clock = 14,018,235,195,990 (880M #Ev/sec/thread) in 238310 samples over 9 threads at 14 Hz (nominal 17 Hz)
# (30x10) : SELECT top,prc,nevs,time,nevents,nsamples,tid,cpulist,scope,code_location FROM prof WHERE event_name = 'task-clock' AND scope like "%user%" ORDER BY top,nevents DESC,time DESC LIMIT 30
```

#Top	Event-%	[Time]	#Events:task-clock	(#Ev/sec)	Samples	TID	CPU-list	Scope	Code location
7	2.75%	48.09s	47,882,352,606	(996M)	814	2495483	1	USER	updecaec_ @ libifs.SP.so (updecaec.F90)
8	2.52%	44.24s	44,176,470,279	(999M)	751	2495703	0	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.43%	42.67s	42,647,058,525	(999M)	725	2495657	5	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.43%	42.67s	42,588,234,996	(998M)	724	2495611	2	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.43%	42.64s	42,588,234,996	(999M)	724	2495675	6	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.42%	42.25s	42,176,470,293	(998M)	717	2495483	1	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.35%	41.35s	41,294,117,358	(999M)	702	2495695	7	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.34%	41.17s	41,117,646,771	(999M)	699	2495641	4	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
8	2.25%	39.55s	39,470,587,959	(998M)	671	2495626	3	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
14	1.55%	27.32s	27,176,470,398	(995M)	462	2495626	3	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.52%	26.90s	26,647,058,637	(991M)	453	2495657	5	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.47%	25.96s	25,823,529,231	(995M)	439	2495675	6	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.47%	25.96s	25,764,705,702	(992M)	438	2495611	2	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.47%	25.85s	25,764,705,702	(997M)	438	2495641	4	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.47%	25.92s	25,705,882,173	(992M)	437	2495703	0	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.42%	24.94s	24,764,705,709	(993M)	421	2495483	1	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
14	1.32%	23.37s	23,117,646,897	(989M)	393	2495695	7	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
20	1.14%	19.79s	19,764,705,744	(999M)	336	2495483	1	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	1.11%	19.50s	19,470,588,099	(999M)	331	2495641	4	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	1.03%	18.07s	18,058,823,403	(999M)	307	2495626	3	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	1.02%	17.84s	17,823,529,287	(999M)	303	2495675	6	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	1.02%	17.84s	17,823,529,287	(999M)	303	2495695	7	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	0.99%	17.31s	17,294,117,526	(999M)	294	2495657	5	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	0.98%	17.25s	17,235,293,997	(999M)	293	2495611	2	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
20	0.98%	17.20s	17,176,470,468	(999M)	292	2495703	0	USER	lascaw_ @ libifs.SP.so (lascaw.F90)
21	1.11%	19.49s	19,470,588,099	(999M)	331	2495657	5	USER	lailt1_ @ libifs.SP.so (lailt1.F90)
21	0.97%	17.07s	17,058,823,410	(1000M)	290	2495695	7	USER	lailt1_ @ libifs.SP.so (lailt1.F90)
21	0.96%	16.90s	16,882,352,823	(999M)	287	2495626	3	USER	lailt1_ @ libifs.SP.so (lailt1.F90)
21	0.96%	16.67s	16,647,058,707	(999M)	283	2495483	1	USER	lailt1_ @ libifs.SP.so (lailt1.F90)
21	0.93%	16.38s	16,352,941,062	(998M)	278	2495703	0	USER	lailt1_ @ libifs.SP.so (lailt1.F90)

ecperf-digest : compute rank#0, basic event group: instructions, particular thread-id

```
# h240.N8+1.T128xt8xhi+ioT16xt8.iomode.od.i16r0w24.0RCA025.275.htco399-1364623.od-rec-basic-17hz/perf-record.dir/aa2-4036/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db -e instructions --where 'tid = 2495483' --limit 30 --verbose
# [...] : Time = 1769s (1769s): Displaying (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs up to 30 rows/output

# [...] : Time = 1769s (1769s): Total number of #Events:instructions = 43,930,081,714,500 (2759M #Ev/sec/thread) in 219000 samples over 9 threads at 13 Hz (nominal 17 Hz)
# (30x10) : SELECT top,prc,mevs,time,nevents,nsamples,tid,cpulist,scope,code_location FROM prof WHERE event_name = 'instructions' AND tid = 2495483 ORDER BY top,nevents DESC,time DESC LIMIT 30
#Top  Event-%      [Time]  #Events:instructions (#Ev/sec) | Samples      TID  CPU-list  Scope      Code location
#----  -
2      6.08%      84.39s      489,150,880,724 ( 5796M ) | 1418      2495483      1      MPI      opal_progress @ libopen-pal.so.40.30.1 (opal_progress.c)
3      5.43%      68.00s      437,431,971,137 ( 6433M ) | 1123      2495483      1      ECCODES      grib_encode_unsigned_longb @ libecodes.so (grib_bits.c)
4      5.33%      73.20s      429,343,047,091 ( 5865M ) | 1219      2495483      1      MPI      uct_mm_iface_progress @ libuct.so.0.0.0 (mm_iface.c)
5      4.89%      68.79s      393,827,402,313 ( 5725M ) | 1152      2495483      1      PTHREAD      pthread_spin_lock @ libpthread-2.28.so (???)
6      3.66%      49.93s      294,409,510,188 ( 5897M ) | 825      2495483      1      USER      updecasc @ libifs.SP.so (updecasc.F90)
7      3.62%      50.56s      291,743,700,551 ( 5770M ) | 845      2495483      1      MPI      uct_rc_mlx5_iface_progress_cyclic @ libuct_ib.so.0.0.0 (ib_mlx5.inl)
8      3.53%      49.98s      284,026,258,725 ( 5683M ) | 829      2495483      1      MPI      ucp_worker_progress @ libucp.so.0.0.0 (ucp_worker.c)
9      3.50%      60.35s      281,589,492,316 ( 4666M ) | 1061      2495483      1      USER      cloudsc @ libifs.SP.so (cloudsc.F90)
10     2.95%      40.54s      237,241,482,448 ( 5852M ) | 680      2495483      1      MPI      opal_timer_linux_get_cycles_sys_timer @ libopen-pal.so.40.30.1 (timer.h)
11     2.38%      32.90s      191,702,122,078 ( 5827M ) | 542      2495483      1      MPI      omp_coll_libnbc_progress @ mca_coll_libnbc.so (coll_libnbc_component.c)
12     1.99%      37.15s      160,019,076,578 ( 4307M ) | 635      2495483      1      USER      rttov_integrate_IP_calc_atmospheric_radiance @ libstrad.SP.so (rttov_integrate.F90)
13     1.99%      26.98s      160,550,129,043 ( 5950M ) | 438      2495483      1      MPI      uct_rc_mlx5_iface_progress_cyclic @ libuct_ib.so.0.0.0 (rc_mlx5.inl)
14     1.66%      22.20s      133,405,300,254 ( 6010M ) | 387      2495483      1      PTHREAD      pthread_self@GLIBC.2.2.5 @ libpthread-2.28.so (???)
15     1.56%      29.79s      125,270,213,219 ( 4205M ) | 513      2495483      1      MATH      __libm_exp10 @ libatlas.so.0.25 (???)
16     1.34%      23.52s      107,711,445,170 ( 4580M ) | 358      2495483      1      USER      laitri_weno @ libifs.SP.so (laitri_weno.F90)
17     1.32%      18.17s      106,007,494,957 ( 5833M ) | 305      2495483      1      MPI      uct_rc_mlx5_iface_progress_cyclic @ libuct_ib.so.0.0.0 (rc_mlx5_iface.c)
18     0.06%      1.69s      4,522,618,901 ( 2680M ) | 29      2495483      1      OMP      kmp_flag_native::notdone_check @ libomp5.so (kmp_wait_release.h)
19     1.23%      22.31s      98,680,777,996 ( 4423M ) | 350      2495483      1      MATH      __libm_powf10 @ libatlas.so.0.25 (???)
20     1.07%      18.63s      85,888,709,177 ( 4611M ) | 298      2495483      1      MATH      __libm_exp10 @ libatlas.so.0.25 (???)
21     1.07%      13.37s      85,827,026,276 ( 6420M ) | 218      2495483      1      ECCODES      grib_ibm_to_long @ libecodes.so (grib_ibmfloat.c)
22     0.90%      22.79s      72,725,350,194 ( 3191M ) | 329      2495483      1      BLAS      mkl_blas_avx2_dgemm_kernel_0_b0_cmr @ libmkl_avx2.so (???)
23     1.02%      12.96s      82,265,186,809 ( 6349M ) | 218      2495483      1      MPI      mca_pml_ucx_recv @ mca_pml_ucx.so (pml_ucx.c)
24     1.02%      30.33s      82,161,074,003 ( 2709M ) | 475      2495483      1      MPI      fldread_mp_fld_read @ libmemogribcouple.V34.ORCA.SP.so (fldread.F90)
25     1.01%      13.86s      81,287,097,039 ( 5866M ) | 238      2495483      1      USER      gpnorm_gfl @ libifs.SP.so (gpnorm_gfl.F90)
26     0.79%      13.98s      63,536,390,099 ( 4543M ) | 243      2495483      1      USER      cloud_satadj @ libifs.SP.so (cloud_satadj.F90)
27     0.89%      11.24s      72,012,742,307 ( 6406M ) | 187      2495483      1      ECCODES      grib_encode_unsigned_longb @ libecodes.so (grib_bits_any_endian.c)
28     0.71%      15.34s      57,470,827,269 ( 3745M ) | 241      2495483      1      USER      rttov_transmit @ libstrad.SP.so (rttov_transmit.F90)
29     0.81%      13.51s      65,085,288,626 ( 4816M ) | 220      2495483      1      USER      lascaw @ libifs.SP.so (lascaw.F90)
30     0.81%      13.97s      64,834,601,780 ( 4640M ) | 215      2495483      1      MATH      __libm_asin10 @ libecodes.so (???)
31     0.72%      12.14s      57,961,593,543 ( 4775M ) | 200      2495483      1      USER      larmes @ libifs.SP.so (larmes.F90)
```


ecperf-digest : compute rank#0, basic event group: instructions, thread-id, user code & linenos

```
# h240.N8+1.T128xt8xh1+ioT16xt8.iomode.od.i16r0w24.0RCA025_Z75.htco399-1364623.od-rec-basic-17hz/perf-record.dir/aa2-4036/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db -e instructions --where 'tid = 2495483 and scope = "USER"' --limit 30 --linenos -v
# [...] : Time = 1769s (1769s): Displaying (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs up to 30 rows/output

# [...] : Time = 1769s (1769s): Total number of #Events:instructions = 43,930,081,714,500 (2759M #Ev/sec/thread) in 219000 samples over 9 threads at 13 Hz (nominal 17 Hz)
# (30x10) : SELECT top,prc,mevs,time,nevents,nsamples,tid,cpulist,scope,code_location FROM profiles WHERE event_name = 'instructions' AND tid = 2495483 and scope = "USER" ORDER BY top,nevents DESC,time DESC LIMIT 30
#Top Event-% [Time] #Events:instructions (#Ev/sec) | Samples TID CPU-list Scope Code Location
#-----
41 0.53% 7.19s 42,336,578,821 ( 5885M ) | 123 2495483 1 USER gpnorm_gfl_ @ libifs.SP.so (gpnorm_gfl.F90:155)
50 0.41% 7.00s 32,667,405,895 ( 4669M ) | 107 2495483 1 USER f_lanint_val @ ifsMASTER.SP (???:0)
55 0.35% 6.59s 28,480,087,561 ( 4321M ) | 117 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsrad.SP.so (rttov_integrate.F90:595)
57 0.34% 5.93s 27,126,058,180 ( 4572M ) | 108 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsrad.SP.so (rttov_integrate.F90:581)
66 0.33% 6.24s 26,532,176,187 ( 4251M ) | 104 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsrad.SP.so (rttov_integrate.F90:576)
70 0.25% 4.54s 20,098,658,992 ( 4430M ) | 71 2495483 1 USER cloudsc_ @ libifs.SP.so (cloudsc.F90:3426)
78 0.26% 3.55s 21,166,436,210 ( 5959M ) | 59 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:280)
79 0.19% 3.51s 14,985,814,610 ( 4274M ) | 65 2495483 1 USER rttov_transmit_ @ libsrad.SP.so (rttov_transmit.F90:343)
80 0.24% 5.50s 19,506,074,771 ( 3545M ) | 69 2495483 1 USER cpcuddh_ @ libifs.SP.so (cpcuddh.F90:294)
81 0.24% 3.14s 18,945,957,584 ( 6031M ) | 52 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:276)
83 0.22% 3.14s 18,038,933,958 ( 5742M ) | 57 2495483 1 USER gpnorm_gfl_ @ libifs.SP.so (gpnorm_gfl.F90:153)
85 0.12% 2.74s 9,678,230,682 ( 3535M ) | 39 2495483 1 USER rttov_transmit_ @ libsrad.SP.so (rttov_transmit.F90:359)
86 0.12% 2.10s 9,737,056,502 ( 4628M ) | 38 2495483 1 USER getkey @ libifsaux.SP.so (drhook.c:1064)
87 0.21% 2.84s 16,822,105,461 ( 5931M ) | 47 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:274)
88 0.21% 2.81s 16,746,182,005 ( 5963M ) | 48 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:278)
89 0.21% 3.74s 16,540,545,394 ( 4428M ) | 58 2495483 1 USER putkey @ libifsaux.SP.so (drhook.c:3145)
90 0.20% 2.76s 15,950,311,201 ( 5783M ) | 44 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:291)
91 0.17% 2.91s 14,063,981,116 ( 4827M ) | 49 2495483 1 USER radiation_two_stream_mp_calc_reflectance_transmittance_sw_ @ libradiation.SP.so (radiation_two_stream.F90:499)
92 0.15% 2.64s 11,894,570,502 ( 4510M ) | 46 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsrad.SP.so (rttov_integrate.F90:578)
94 0.19% 2.60s 15,068,098,631 ( 5789M ) | 40 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:282)
95 0.19% 2.60s 14,977,638,609 ( 5759M ) | 43 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:289)
96 0.18% 2.44s 14,451,037,962 ( 5920M ) | 40 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:272)
97 0.18% 2.38s 14,337,061,329 ( 6020M ) | 39 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:268)
98 0.16% 2.58s 12,926,730,298 ( 5004M ) | 44 2495483 1 USER larmes_ @ libifs.SP.so (larmes.F90:553)
100 0.17% 2.32s 13,674,021,317 ( 5905M ) | 39 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:284)
102 0.15% 3.10s 12,211,971,223 ( 3941M ) | 48 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsrad.SP.so (rttov_integrate.F90:605)
103 0.17% 2.89s 13,356,158,201 ( 4619M ) | 42 2495483 1 USER lautri_weno_ @ libifs.SP.so (lautri_weno.F90:786)
104 0.17% 2.26s 13,314,212,525 ( 5878M ) | 37 2495483 1 USER updecaec_ @ libifs.SP.so (updecaec.F90:286)
106 0.16% 2.19s 13,140,167,152 ( 6013M ) | 36 2495483 1 USER gpnorm_gfl_ @ libifs.SP.so (gpnorm_gfl.F90:154)
107 0.12% 2.40s 9,797,118,396 ( 4084M ) | 36 2495483 1 USER cpg_ @ libifs.SP.so (cpg.F90:897)
```

ecperf-digest : compute rank#0, basic event group: instructions, thread-id, user, with source code annotation

```
# h240.N8+1.T128xt8xh1+ioT16xt8.iomode_od.i16r0w24.0RCA025.Z75.htco399-1364623.od-rec-basic-17hz/perf-record.dir/aa2-4036/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
ecperf-digest ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db -e instructions --where 'tid = 2495483 and scope = "USER"' --limit 4 --verbose \
--annotate --searchpath=$HOME/git/naih_CY47R3_RAPS20
# [...] : Time = 1769s (1769s): Displaying (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs up to 4 rows/output

# [...] : Time = 1769s (1769s): Total number of #Events:instructions = 43,930,081,714,500 (2759M #Ev/sec/thread) in 219000 samples over 9 threads at 13 Hz (nominal 17 Hz)
# (4x10) : SELECT top,prc,mevs,time,nevents,nsamples,tid,cpulist,scope,code location FROM profiles WHERE event_name = 'instructions' AND tid = 2495483 and scope = "USER" ORDER BY top,nevents DESC,time DESC LIMIT 4
#Top Event-% [Time] #Events:instructions (#Ev/sec) | Samples TID CPU-list Scope Code location
#-----
#41 0.53% 7.19s 42,336,578,821 ( 5885M ) | 123 2495483 1 USER gpnorm_gfl_ @ libifs.SP.so (gpnorm_gfl.F90:155)

    *** /home/mps/git/naih_CY47R3_RAPS20/ifs/utility/gpnorm_gfl.F90 near line 155
    | DO JL=IST,IEND
    | ZGP(JL,IFLD,IBL) = ZGP(JL,IFLD,IBL)+GFL(JL,JF,JGFL,IBL)
#155 | ZMINBLK(IFLD,IBL) = MIN(ZMINBLK(IFLD,IBL),GFL(JL,JF,JGFL,IBL))
    | ZMAXBLK(IFLD,IBL) = MAX(ZMAXBLK(IFLD,IBL),GFL(JL,JF,JGFL,IBL))
    | ENDDO
    | ENDDO
    | ENDDO

50 0.41% 7.00s 32,667,405,895 ( 4669M ) | 107 2495483 1 USER f_lanint_val @ ifsMASTER.SP (??? :0)
55 0.35% 6.59s 28,480,087,561 ( 4321M ) | 117 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsatrads.SP.so (rttov_integrate.F90:595)

    *** /home/mps/git/naih_CY47R3_RAPS20/satrads/rttov/main/rttov_integrate.F90 near line 595
    | #ifdef PARKIND1_SINGLE
    | IF (tau_level > min_tau) THEN
#595 | #endif
    | auxrad_stream%down(layer,ist,i) = auxrad_stream%down(layer-1, ist, i) + &
    | fac1 * &
    | fac2_thermal_path1 * &
    | ((dtau_layer * &
    |
57 0.34% 5.93s 27,126,058,180 ( 4572M ) | 108 2495483 1 USER rttov_integrate_IP_calc_atmospheric_radiance_ @ libsatrads.SP.so (rttov_integrate.F90:581)

    *** /home/mps/git/naih_CY47R3_RAPS20/satrads/rttov/main/rttov_integrate.F90 near line 581
    | (dtau_layer * &
    | (auxrad%air(layer,i) + &
#581 | daux_layer * &
    | od_singlelayer_r) - &
    | daux_layer * &
    | tau_level)
    | IF (do_lambertian(i)) THEN ! Lambertian reflected downwelling option no linear-in-tau
```

ecperf-digest : compute rank#0, basic event group:
cycles, thread-id linenos & calltrees

```

#240.N8+1.T128xt8xh1+ioT16xt8.iodmode_od.16r0w+ORCA025.275.btc0399-1364623.od-rec-basic-17Hz/perf-record.dir/aa2-4036/xxe-ifzMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.gz
#cpperf-digest ecperrf-exe-ifzMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db -o cycles --where 'tid = 2495483' --limit 1 --linenos --calltree -v
# [...] : Time = 1769s (1769s): Displaying (per-TID, ntids=9, #Events=sorted) records with # of samples >= 1 and time >= 0.000000 secs up to 1 rows/output

# [...] : Time = 1769s (1769s): Total number of #Events:cycles = 43,160,625,032,310 (2711M #Ev/sec/thread) in 237619 samples over 9 threads at 14 Hz (nominal 17 Hz)
# (ix10) : SELECT top,prc,mvts,time,nevents,nsamples,tid,cpulist,scope,code_location FROM callprofiles WHERE event_name = 'cycles' And tid = 2495483 ORDER BY top,nevents DESC,time DESC LIMIT 1
#Top Event-% [Time] #Events:cycles (#Ev/sec) | Samples TID CPU-list Scope Code location
-----
1 0.17% 2.82s 8,954,428,172 ( 3177M ) | 46 2495483 1 KERNEL unmap_page_range @ ??? (???:0)
| {KERNEL} unmap_page_range @ ??? (???:0)
| {KERNEL} zap_page_range_single @ ??? (???:0)
| {KERNEL} unmap_mapping_pages @ ??? (???:0)
| {LUSTRE} vvp_conf_set @ lustre.ko (???:0)
| {LUSTRE} cl_conf_set @ obdclass.ko (???:0)
| {LUSTRE} ll_layout_conf @ lustre.ko (???:0)
| {LUSTRE} ll_md_blocking_ast @ lustre.ko (???:0)
| {LUSTRE} ldlm_cancel_callback @ ptlrpc.ko (???:0)
| {LUSTRE} ldlm_cli_cancel_local @ ptlrpc.ko (???:0)
| {LUSTRE} ldlm_cli_cancel_list_local @ ptlrpc.ko (???:0)
| {LUSTRE} ldlm_prep_elc_req @ ptlrpc.ko (???:0)
| {LUSTRE} ldlm_prep_enqueue_req @ ptlrpc.ko (???:0)
| {MDC} mdc_intent_getattr_pack.isra.15 @ ??? (???:0)
| {MDC} mdc_enqueue_base @ ??? (???:0)
| {MDC} mdc_intent_lock @ ??? (???:0)
| {LUSTRE} lmv_intent_remote.isra.8 @ lmv.ko (???:0)
| {LUSTRE} lmv_intent_lookup @ lmv.ko (???:0)
| {LUSTRE} lmv_intent_lock @ lmv.ko (???:0)
| {LUSTRE} ll_lookup_it @ lustre.ko (???:0)
| {LUSTRE} ll_lookup_md @ lustre.ko (???:0)
| {KERNEL} __lookup_slow @ ??? (???:0)
| {KERNEL} lookup_slow @ ??? (???:0)
| {KERNEL} walk_component @ ??? (???:0)
| {KERNEL} link_path_walk @ ??? (???:0)
| {KERNEL} path_lookupat.isra.49 @ ??? (???:0)
| {KERNEL} filename_lookup.part.65 @ ??? (???:0)
| {KERNEL} vfs_statx @ ??? (???:0)
| {KERNEL} __do_sys_newstat @ ??? (???:0)
| {KERNEL} __do_syscall.64 @ ??? (???:0)
| {KERNEL} entry_SYSCALL.64.after_buframe @ ??? (???:0)
| {LIBC} __xstat64 @ libc-2.28.so (???:0)
| {MPI} iom_mpi_om_open @ libnemogribcoup.V34_ORCA.SP.so (iom.F90:265)
| {MPI} limrst_2_mpi_lm_rst_om_2 @ libnemogribcoup.V34_ORCA.SP.so (limrst_2.F90:97)
| {MPI} sbcice_lin_2_mpi_sbc_ice_lin_2 @ libnemogribcoup.V34_ORCA.SP.so (sbcice_lin_2.F90:191)
| {MPI} sbcmcd_mpi_sbc_ @ libnemogribcoup.V34_ORCA.SP.so (sbcmcd.F90:391)
| {MPI} step_mpi_step @ libnemogribcoup.V34_ORCA.SP.so (step.F90:129)
| {MPI} nemogrcoup_step @ libnemogribcoup.V34_ORCA.SP.so (nemogrcoup_step.F90:69)
| {USER} couplnemo @ libifs.SP.so (couplnemo.F90:190)
| {USER} cnt4 @ libifs.SP.so (cnt4.F90:1269)
| {USER} cnt3 @ libifs.SP.so (cnt3.F90:199)
| {USER} cnt2 @ libifs.SP.so (cnt2.F90:110)
| {USER} cnt1 @ libifs.SP.so (cnt1.F90:189)
| {USER} cnt0 @ libifs.SP.so (cnt0.F90:210)
| {USER} MAIN_ @ ifzMASTER.SP (master.F90:231)
| {USER} main @ ifzMASTER.SP (???:0)
| {LIBC} libc_start_main @ libc-2.28.so (???:0)

```

Working with SQLite databases

- The `ecperf-digest` by default converts output from `ecperf script` to a SQLite (version 3) database
- On Bologna systems, please module-load the latest version: **module load sqlite**
- SQLite is a server-less database and has been developed since year 2000
- It has gained a huge popularity and has got 100's of millions of installations
- You can create SQLite databases using f.ex. one of the following tools:
 - Using its own handy CLI or command line interface (command `sqlite3`)
 - Perl, Python, PHP, Java
 - C/C++ (and thus Fortran and many other languages)
- SQL query output directly into various table-formats, `csv`, `Excel`, `JSON`, ...
- The same database (a single file) can be used on Unix, Windows and Mac systems w/o conversion
- More information about SQLite from <https://www.sqlite.org>

Accessing extracted `ecperf`-data directly with SQLite

- Much of the output shown in the previous slides can be repeated by using directly the SQLite CLI (`sqlite3`)
- You can use the predefined queries (see commands **.tables** and **.schema**)
- You can change the output mode with `.mode` and column alignments (`.width`)
- “Research” more in detail the metadata-TABLEs, especially **mpi**, **catalog** and **metadata**
- You can display every single sample (measurement) that **ecperf-record** produced and perhaps plot them
- There are also summaries provided by the VIEWS like **event**, **pertid**, **percpu**, and **grandsummary**
- You won’t however be able to embed (annotate) with source code – only the **ecperf-digest**-interface can
 - Also code locations are always replicated
- A few examples to follow ...

SQLite with ecperf-database [1]

- To avoid accidental overwrites, always open SQLite databases in these examples with the **-readonly** flag
- You can configure your desired defaults in \$HOME/.sqliterc
- The following example shows how to display the event summaries

```
sqlite3 -readonly ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364585.freq-17Hz.db
-- Loading resources from /home/mps/.sqliterc
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sql> .mode box # this is my default anyway
sql> select * from event;
select * from event;
```

id	name	nevents	nsamples	mpirank
1	cycles	48688750535932	307847	0
2	instructions	48034744285871	286592	0
3	L1-dcache-loads	15155887402478	191476	0
4	L1-dcache-load-misses	336280395393	219977	0
5	cache-references	607573787054	201700	0
6	dTLB-load-misses	4785525204	161945	0
7	branch-misses	18376820025	177611	0
8	branch-instructions	5436200100903	190182	0
9	task-clock	18147588108261	308509	0

```
Run Time: real 0.001 user 0.000230 sys 0.000000
sql> .quit # .. or <ctrl-D>
```

SQLite with ecperf-database [2]

- Lets see what's in the TABLE **mpi** – contains one row only
- Changing display mode to **line** for much clearer view

```
sqlite3 -readonly ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364585.freq-17Hz.db
-- Loading resources from /home/mps/.sqliterc
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sql> .mode line
sql> select * from mpi;
select * from mpi;
rank = 0
hostname = aal-4060.bullx
user = mps
cpudesc = AMD EPYC 7742 64-Core Processor
omp_places = 1,2,3,4,5,6,7,0
omp_threads = 8
num_threads = 10
num_event_ids = 9
pid = 1589648
tidlist = 1589648,1589709,1589719,1589745,1589763,1589779,1589795,1589812,1589828,1589836
cpulist = 0-7
captured = 2021-11-22 11:48:55.000000
captured_epoch = 1637574535.000000
reference = 2021-11-22 11:10:43.301627
reference_epoch = 1637579443.301627
reference_clock = 419234.2335880067
runtime = 2287.5735699401
sampling_delay = 0.321386993979104
sampling_frequency = 17
sampling_time = 2287.252182
sampling_begin_clock = 419234.554975
sampling_end_clock = 421521.807158
database_filename = /zfs1/mps/BLQ/perffi/22nov2021/hres_tco399/intel.blq.hpcx/ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364585.freq-17Hz.db
executable = /lus/h2res01/scratch/mps/build-20/hpcx.intel.blq.sp/flexbuild/bin/ifsMASTER.SP
perfdata_filename = /ec/res1/scratch/mps/rundir/RAPS/20/tco3991137/hloa/hres/intel.blq.hpcx/ <... truncated ...> htco399-1364585.od-rec-hpc-17hz/perf-record.dir/aal-4060/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364585.freq-17Hz.data
perfdata_filesize = 964705604
report_filename = /zfs1/mps/BLQ/perffi/22nov2021/hres_tco399/intel.blq.hpcx/ <... truncated ...> htco399-1364585.od-rec-hpc-17hz/perf-record.dir/aal-4060/exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364585.freq-17Hz.gz
report_filesize = 155987118
report_datasize = 2249923129
max_scopelen = 9
perf_event_paramid = -1
kptr_restrict = 0
nmi_watchdog = 0
Run Time: real 0.000 user 0.000199 sys 0.000000
```

SQLite with ecperf-database [3]

- Lets show a DrHook-like output – the mode needs to be set to **column**
- Please note that there is *always* replication of the code locations
- Using now the database `ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db`
- Using nearly the same SQL-query since sorting (**ORDER BY**) already defined in the **VIEW prof**

```
sqlite3 -readonly ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364623.freq-17Hz.db
```

```
sql> .mode column # best mode when displaying calling trees
```

```
sql> .width 0 # reset column width to left-justified
```

```
sql> select * from prof where event_name = 'task-clock' and scope like "%user%" limit 30;
```

```
select * from prof where event_name = 'task-clock' and scope like "%user%" limit 30;
```

best	top	event_id	event_name	prc	mevs	time	nevents	nsamples	tid	cpulist	scope	code_location
*	7	3	task-clock	2.75	995.741	48.087147	47882352606	814	2495483	1	USER	updecaec_ @ libifs.SP.so (updecaec.F90)
*	8	3	task-clock	2.52	998.539	44.241122	44176470279	751	2495703	0	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.43	999.436	42.671129	42647058525	725	2495657	5	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.43	998.679	42.644565	42588234996	724	2495675	6	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.43	998.173	42.666186	42588234996	724	2495611	2	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.42	998.373	42.24521	42176470293	717	2495483	1	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.35	998.728	41.3467	41294117358	702	2495695	7	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.34	998.686	41.171766	41117646771	699	2495641	4	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
	8	3	task-clock	2.25	998.028	39.548564	39470587959	671	2495626	3	USER	cloudsc_ @ libifs.SP.so (cloudsc.F90)
*	14	3	task-clock	1.55	994.727	27.320531	27176470398	462	2495626	3	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.52	990.645	26.898705	26647058637	453	2495657	5	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.47	994.693	25.961305	25823529231	439	2495675	6	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.47	996.563	25.853573	25764705702	438	2495641	4	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.47	992.493	25.959572	25764705702	438	2495611	2	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.47	991.91	25.915542	25705882173	437	2495703	0	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.42	993.073	24.937439	24764705709	421	2495483	1	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
	14	3	task-clock	1.32	989.343	23.366671	23117646897	393	2495695	7	USER	rttov_integrate_IP_calc_atmospheric_radiance_ @ libstrad.SP.so (rttov_integrate.F90)
*	20	3	task-clock	1.14	998.695	19.790542	19764705744	336	2495483	1	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	1.11	998.71	19.495743	19470588099	331	2495641	4	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	1.03	999.314	18.071217	18058823403	307	2495626	3	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	1.02	999.114	17.839338	17823529287	303	2495675	6	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	1.02	999.266	17.836627	17823529287	303	2495695	7	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	0.99	998.818	17.314588	17294117526	294	2495657	5	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	0.98	999.006	17.25244	17235293997	293	2495611	2	USER	lascav_ @ libifs.SP.so (lascav.F90)
	20	3	task-clock	0.98	998.509	17.202127	17176470468	292	2495703	0	USER	lascav_ @ libifs.SP.so (lascav.F90)
*	21	3	task-clock	1.11	999.1	19.488129	19470588099	331	2495657	5	USER	lailti_ @ libifs.SP.so (lailti.F90)
	21	3	task-clock	0.97	999.511	17.067171	17058823410	290	2495695	7	USER	lailti_ @ libifs.SP.so (lailti.F90)
	21	3	task-clock	0.96	999.142	16.896843	16882352823	287	2495626	3	USER	lailti_ @ libifs.SP.so (lailti.F90)
	21	3	task-clock	0.96	998.677	16.669117	16647058707	283	2495483	1	USER	lailti_ @ libifs.SP.so (lailti.F90)
	21	3	task-clock	0.93	998.414	16.378914	16352941062	278	2495703	0	USER	lailti_ @ libifs.SP.so (lailti.F90)

```
Run Time: real 1.917 user 1.668735 sys 0.248438
```


SQLite with ecperf-database [4]

- Finally, some forensics !
- Looking at the few first samples page-faults (event **faults**) and where they occurred
- Using now the database ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364684.freq-17Hz.db
- Columns one and two look pretty much ready for a plotting program : time versus number of events
- As you can see this is an expert query !

```
sqlite3 -readonly ecperf-exe-ifsMASTER.SP.mpi-0.omp-8.jobid-1364684.freq-17Hz.db
```

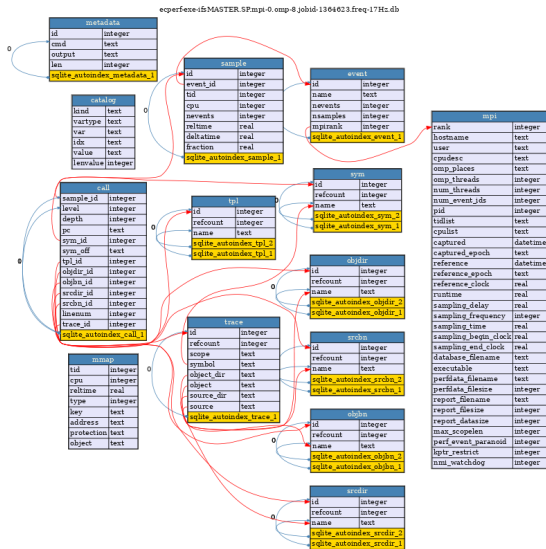
```
sql> .mode box
sql> select retime,nevents,tid,scope,symbol,source_dir,source,linenum \
  from sample,call,trace \
  where event_id = (select id from event where name = 'faults') and \
  sample_id = sample_id and level = 1 and trace_id = trace_id \
  limit 50,20;
```

retime	nevents	tid	scope	symbol	source_dir	source	linenum
8.63741614	1	707032	USER	getenv_@plt		NULL	NULL
8.63743018	1	707032	USER	getenv_		NULL	NULL
8.63744818	159	707032	USER	getenv_		NULL	NULL
8.686211458	13346095	707032	LD	memset		NULL	NULL
10.158193656	1	707184	USER	ec_meminfo_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/ifsaux/utilities	ec_meminfo.F90	248
10.158199485	1	707184	LD	_dl_runtime_resolve_xsavec		NULL	NULL
10.158209665	1	707184	LD	do_lookup_x		NULL	NULL
10.158233745	1114	707184	LD	do_lookup_x		NULL	NULL
12.524036158	540	707152	USER	sutrl_mod_mp_sutrl_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/trans/module	sutrl_mod.F90	258
12.547291809	548	707200	USER	suleg_mod_mp_suleg_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/trans/module	suleg_mod.F90	756
12.547327919	2226	707168	USER	suleg_mod_mp_suleg_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/trans/module	suleg_mod.F90	756
12.547839918	2135	707217	USER	suleg_mod_mp_suleg_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/trans/module	suleg_mod.F90	1037
12.549852824	2212	707233	USER	suleg_mod_mp_suleg_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/trans/module	suleg_mod.F90	1037
15.241363112	2147	707152	USER	mpdecomp_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/wam/wam_oper	mpdecomp.F90	2499
15.523626409	8738318	707184	LIBC	__memset_avx2_unaligned_erms		NULL	NULL
16.588614218	384	707200	USER	iostream_mix_mp_grid_in_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/ifs/module	iostream_mix.F90	2972
16.590396455	2077	707248	USER	iostream_mix_mp_grid_in_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/ifs/module	iostream_mix.F90	2972
16.591088983	616	707200	USER	iostream_mix_mp_grid_in_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/ifs/module	iostream_mix.F90	2972
16.597081964	3658	707200	USER	sugridug_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/ifs/setup	sugridug.F90	227
16.598108312	672	707168	USER	sugridug_	/etc/ecmwf/ssd/ssd1/tmpdirs/mps.1362310/hpcx.intel.blq.sp/ifs/setup	sugridug.F90	221

```
Run Time: real 0.000 user 0.000347 sys 0.000174
```

Supplementary slides

Entity relationship diagram (ERD)



Important parameters for `ecperf`-tools

- `ecperf`-tools are primarily meant to be used on Bologna compute nodes (parallel partition)
- By default we have access to user (program) symbols only, and will either miss kernel symbols, or they are displayed in hexadecimal
- In order to resolve (most of the) kernel symbols from their hexadecimal addresses, the following kernel files need to contain the following values:

Kernel file	Value
<code>/proc/sys/kernel/perf_event_paranoid</code>	-1
<code>/proc/sys/kernel/kptr_restrict</code>	0
<code>/proc/sys/kernel/nmi_watchdog</code>	0

- These node-wise values can only be changed with root-privileges
- There is a script on Bologna-system that allows designated users to modify these on the allocated compute nodes

Should I pay attention to the compiler options ? [1]

- Using `ecperf` tools (or `perf` for that matter), usually means you don't need to pay attention to how you compile your code
- These tools are capable of obtaining information no matter what
- However, on very high optimization levels (`-O3` or higher), and omitting symbol information (not supplying `-g` or similar) may lead to a situation where only hexadecimal addresses (program counters) are displayed next to the library and/or executable name (if any)
- Also calling tree (call graphs) maybe broken or partially incomplete when calling functions (say) within OpenMP regions
- You may also see instantaneous (or orphaned) invocations of some library functions, often with the Intel MKL-library
this is due to high optimization levels used to build those libraries

Should I pay attention to the compiler options ? [2]

- Call graphs will be more accurate if your code is compiled with `-fno-omit-frame-pointer`, however
 - Intel compiler adopts `-fno-omit-frame-pointer` by default, if `-g` or similar was used
 - GNU compiler requires explicit use of `-fno-omit-frame-pointer` option during compilation even if `-g` or similar was used
- Whilst runtime performance penalty of binaries compiled with `-fno-omit-frame-pointer` is unnoticeable (for IFS at least), most of the application libraries unfortunately are indeed omitting the frame pointer, and this has lead to broken calling tree, for example with MKL-library calls

Should I pay attention to the compiler options ? [3]

- Fortunately `ecperf-record` enables to use different strategies for obtaining call graphs depending whether the frame pointer is omitted or not
- The following table illustrates options or environment variables needed to fix the situation

Compiler flag	export <code>PERF_GRAPH=<type></code> or option <code>--graph=<type></code>
<code>-fomit-frame-pointer</code>	<code>dwarf</code>
<code>-fno-omit-frame-pointer</code>	<code>fp</code>

- The current default in `ecperf-record` is `fp` i.e. assumes direct or implicit use of `-fno-omit-frame-pointer`
- You can also mix and match these two values, `dwarf` and `fp`, and don't be surprised by different calling trees
- Genuine Intel CPUs (since Haswell) also support the value of `lbr` (**l**ast **b**ranch **r**ecord)

ecperf-record quick usage

- Usage: `ecperf-record [options] – executable [options]`
- Some env. vars to control `ecperf-record` (some can be passed as args):

Description	Environment	Argument	Default	Values
MPI-rank number		<code>--mpirank=<number></code>	Automatic	
<code>ecperf</code> start delay (ms)	<code>PERF_DELAY=<msecs></code>	<code>-d <msecs> --delay=<msecs></code>	0	2s = 2000ms = 2000
Sampling frequency	<code>PERF_FREQ=<hz></code>	<code>-F <hz> --frequency=<hz></code>	17	
	<code>PERF_HZ=<hz></code>	<code>--hz=<hz></code>	17	
Performance groups	<code>PERF_GROUPS=<grp></code>	<code>-g <grp> --groups=<grp></code>	basic	cache,branch,tlb,hpc,flops,idle
Performance events	<code>PERF_EVENTS=<ev></code>	<code>-e <ev> --events=<ev></code>	cycles,instructions,task-clock	See <code>ecperf list</code>
Call-graph type	<code>PERF_GRAPH=<type></code>	<code>--graph=<type></code>	fp	dwarf or Intel-only: lbr
Maxsize for a <code>perf.data</code> -file	<code>PERF_MAXSIZE=<size></code>	<code>-m <size> --maxsize=<size></code>	100G (per task)	100000000000 = 100G
Ranks to be monitored	<code>PERF_RANKS=<keys></code>	<code>-r <key> --ranks=<key></code>	persocket	pernode, all, <begin>:<inc>:<end> node# with # = \$SLURM_NODEID
Keep <code>perf.data</code> -files	<code>PERF_DATA_KEEP=yes</code>	<code>--keep</code>	no	
Remove <code>perf.data</code> -files	<code>PERF_DATA_KEEP=no</code>	<code>--nokeep</code>	no	
Set <code>/proc/sys/kernel</code> values	<code>PERF_BE_PARANOID=yes</code>	<code>--paranoid</code>	yes	
Don't set <code>/proc/sys/kernel</code> values	<code>PERF_BE_PARANOID=no</code>	<code>--noparanoid</code>	yes	